JavaFX Layouts

Layouts are the top-level container classes that define the UI styles for scene graph objects. Layout can be seen as the parent node to all the other nodes. JavaFX provides various layout panes that support different styles of layouts.

We have several built-in layout panes in JavaFX that are HBox, VBox, StackPane, FlowBox, AnchorPane, etc.

All these classes belong to javafx.scene.layout package. javafx.scene.layout.Pane class is the base class for all the built-in layout classes in JavaFX.

Layout Classes

javafx.scene.layout Package provides various classes that represents the layouts. The classes are described in the table below.

Class	Description
BorderPane	Organizes nodes in top, left, right, centre and the bottom of the screen.
FlowPane	Organizes the nodes in the horizontal rows according to the available horizontal spaces. Wraps the nodes to the next line if the horizontal space is less than the total width of the nodes
GridPane	Organizes the nodes in the form of rows and columns.
HBox	Organizes the nodes in a single row.
Pane	It is the base class for all the layout classes.
StackPane	Organizes nodes in the form of a stack i.e. one onto another
VBox	Organizes nodes in a vertical column.

Steps to create layout

In order to create the layouts, we need to follow the following steps.

1. Instantiate the respective layout class, for example, HBox root = new HBox();

2. Setting the properties for the layout, for example, root.setSpacing(20);

3. Adding nodes to the layout object, for example, root.getChildren().addAll(<NodeObjects>);

JavaFX UI Controls

The graphical user interface of every desktop application mainly considers UI elements, layouts and behaviors. The UI elements are the one which are actually shown to the user for interaction or information exchange. Layout defines the organization of the UI elements on the screen. Behavior is the reaction of the UI element when some event is occurred on it.

However, the package javafx.scene.control provides all the necessary classes for the UI components like Button, Label, etc. Every class represents a specific UI control and defines some methods for their styling.

SN	Control	Description
1	Label	Label is a component that is used to define a simple text on the screen. Typically, a label is placed with the node, it describes.
2	Button	Button is a component that controls the function of the application. Button class is used to create a labelled button.
3	RadioButton	The Radio Button is used to provide various options to the user. The user can only choose one option among all. A radio button is either selected or deselected.

4	CheckBox	Check Box is used to get the kind of information from the user which contains various choices. User marked the checkbox either on (true) or off(false).
5	TextField	Text Field is basically used to get the input from the user in the form of text. javafx.scene.control.TextField represents TextField
6	PasswordField	PasswordField is used to get the user's password. Whatever is typed in the passwordfield is not shown on the screen to anyone.
7	HyperLink	HyperLink are used to refer any of the webpage through your appication. It is represented by the class javafx.scene.control.HyperLink
8	Slider	Slider is used to provide a pane of options to the user in a graphical form where the user needs to move a slider over the range of values to select one of them.
9	ProgressBar	Progress Bar is used to show the work progress to the user. It is represented by the class javafx.scene.control.ProgressBar.
10	ProgressIndicator	Instead of showing the analogue progress to the user, it shows the digital progress so that the user may know the amount of work done in percentage.
11	ScrollBar	JavaFX Scroll Bar is used to provide a scroll bar to the user so that the user can scroll down the application pages.
12	Menu	JavaFX provides a Menu class to implement menus. Menu is the main component of any application.
13	ToolTip	JavaFX ToolTip is used to provide hint to the user about any component. It is mainly used to provide hints about the text fields or password fields being used in the application.

Introduction to JavaFX

JavaFX is a set of graphics and media packages that enables developers to design, create, test, debug, and deploy rich client applications that operate consistently across diverse platforms.

- With JavaFX, you can build many types of applications.
- Typically, they are network-aware applications that care deployed across multiple platforms and display information in a high -performance modern user interface that features audio, video, graphics, and animations.

The JavaFX GUI Programming

- GUI of an application is made up of Windows (Called Sage in JavaFX)
- A window has a container (called Scene) to host the UI root layout container
- UI components are first added to a root layout container (such as VBox) then place in the Scene
- UI Components raise Events when the user interacts with them (such as a MouseClicked event is raised when a button is clicked)
- Programmer write Event Handlers to respond to the UI events

We must have to know about the following terminologies

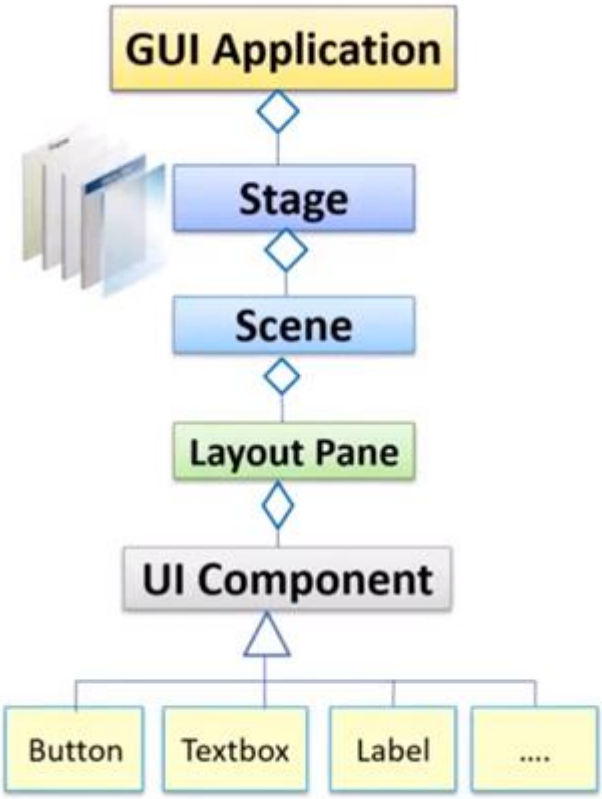
- **Stage**
- **Layout**
- **Scene**
- **Controls**
- **Events**

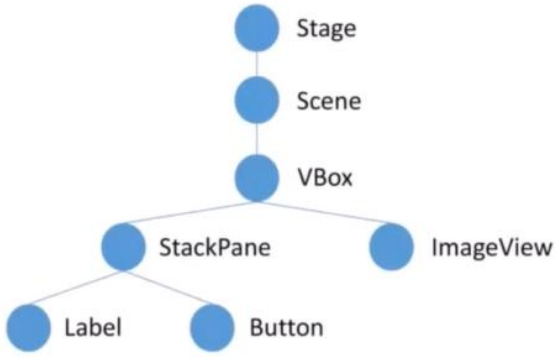
Stage = Windows where a Scene I displayed

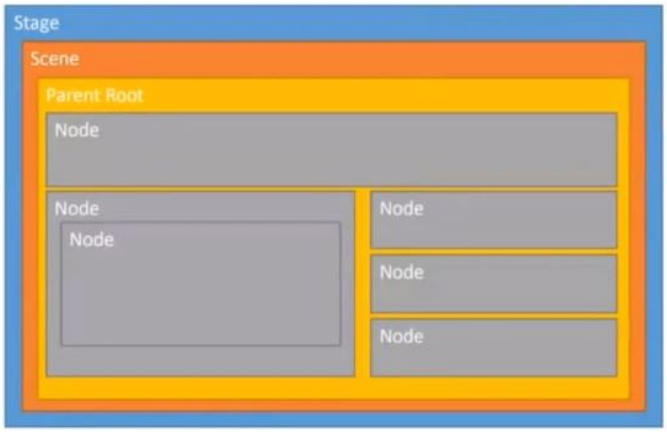
Scene= Container to host the UI root layout container

Stage Methods

```
primaryStage.show()
primaryStage.setScene(sc)
primaryStage.setTitle(value)
primaryStage.getTitle()
primaryStage.setWidth(value)
primaryStage.getWidth()
primaryStage.setHeight(value)
primaryStage.getHeight()
primaryStage.setFullScreen(true)
primaryStage.isFullScreen()
primaryStage.close()
```



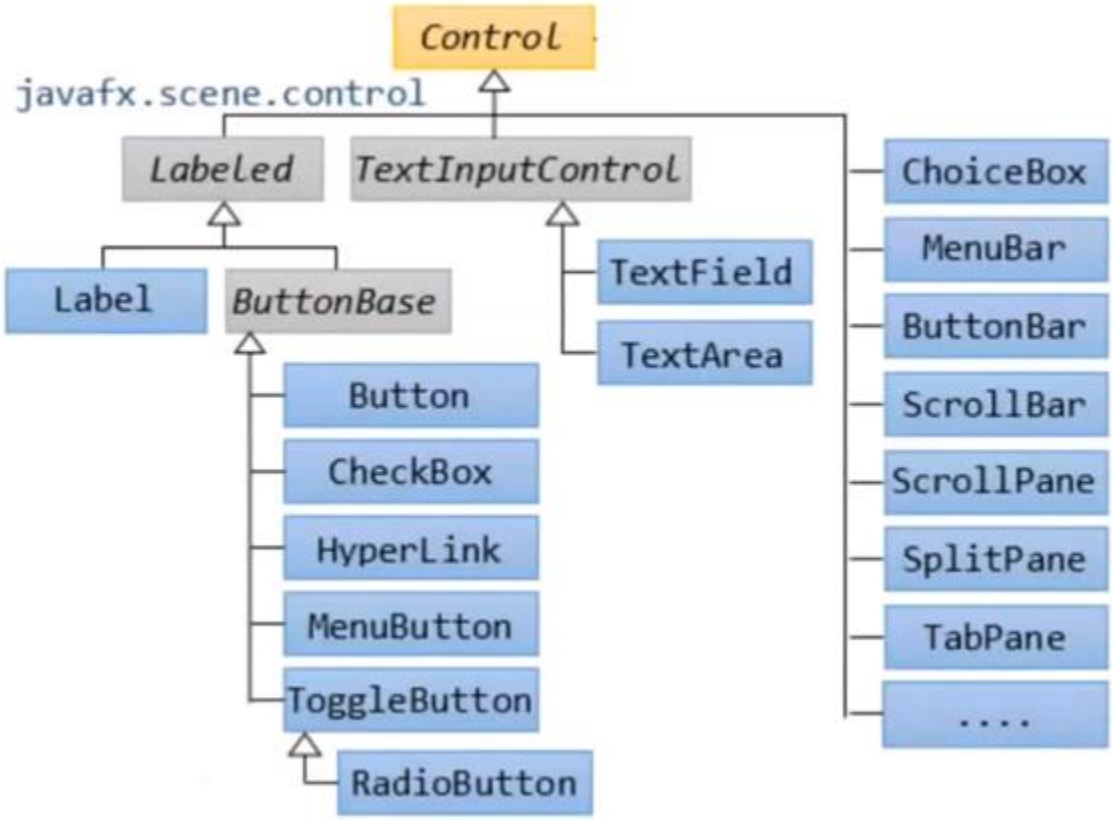




What Makes up JavaFX?

- UI Components
  - Set of pre-built UI components that can be composed to create GUI
  - Ex. Buttons, text-fields, menu, tables, lists, etc.
- Layout containers
  - Control placement/positioning of components in the form of (e.g VBox and HBox)

JavaFX UI Components Hierarchy



Creating JavaFX GUI Application

- Import javafx.application.Application class
- Create a class that extends javafx.application.Application
- Override the start method in the Application class

For example,

```
import javafx.application.Application;
import javafx.stage.Stage;
public  class App extends Application{
      @Override
      public void start(Stage stage){
            stage.setTitle("My First App");
            stage.show();
      }
      public static void main(String args[]){
            launch(args);
      }
}
```

JavaFX program to display a button

```
import javafx.application.Application;
import javafx.scene.layout.*;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.stage.Stage;
public class ButtonApp extends Application{
	@Override
	public void start(Stage stage){
		stage.setTitle("Button App");
		//creating controls
		Button btn1= new Button("Click Me");

		//defining layouts
		HBox h = new HBox();
		//adding components to layout
		h.getChildren().add(btn1);

		//creating scene
		Scene sc = new Scene(h);

		//setting scene to stage
		stage.setScene(sc);

		//display stage
		stage.show();
	}
	public static void main(String args[]){
		launch(args);
	}
}
```

Layout

Layout containers or panels can be used to allow for flexible and dynamic arrangements of the UI controls within a scene graph of a JavaFX application. The JavaFX Layout API includes the following container classes that automate common layout models:

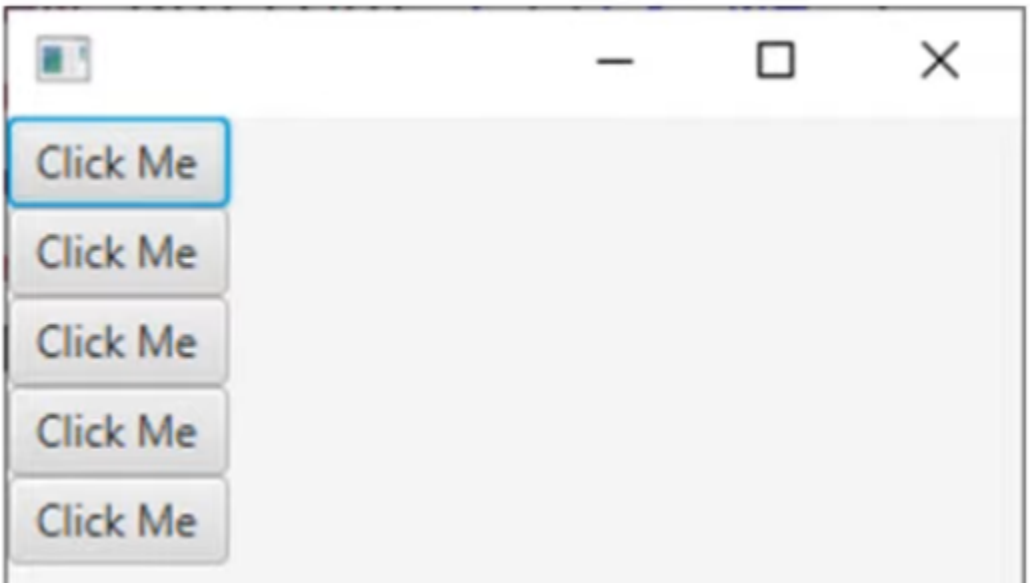
- The **HBox** class arranges its content nodes horizontally in a single row.
- The **VBox** class arranges its content nodes vertically in a single column.
- The **StackPane** class places its content nodes in a back-to-front single stack.

- The **FlowPane** class arranges its content nodes in either horizontal or vertical "flow", wrapping at the specified width (for horizontal) or height (for vertical) boundaries.
- The **GridPane** class enables that developer to create a flexible grid of rows and columns in which to lay out content nodes.
- The **BorderPane** class lays out its content nodes in the top, bottom, right, left, or center region.

HBox

```
// create a HBox
HBox hBox = new HBox();

// create a label
Label label = new Label("this is HBox example");

// add label to hBox
hBox.getChildren().add(label);

// add buttons to hBox
for (int i = 0; i < 5; i++) {
      hBox.getChildren().add(new Button("Button " + (int)(i + 1)));
}
```

VBox



```
// create a VBox
VBox vBox = new VBox();

// create a label
Label label = new Label("this is VBox example");

// add label to vBox
vBox.getChildren().add(label);

// add buttons to vBox
for (int i = 0; i < 5; i++) {
      vBox.getChildren().add(new Button("Button " + (int)(i + 1)));
}
```

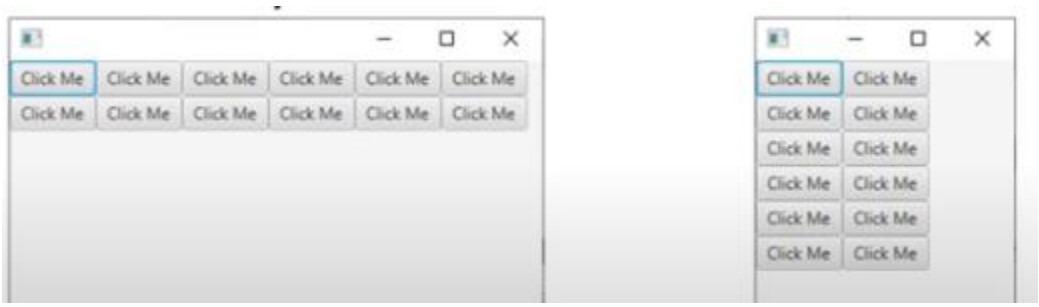
StackPane

The StackPane layout pane places all the nodes into a single stack where every new node gets placed on the top of the previous node.

```
Button btn1 = new Button("Button 1 on bottom ");
Button btn2 = new Button("Button 2 on top");
StackPane root = new StackPane();
root.getChildren().addAll(btn1,btn2);
```

FlowPane

FlowPane layout panel organizes the nodes in a flow that are wrapped at the flow pane's boundary

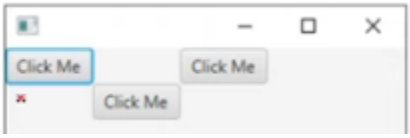



```
FlowPane root = new FlowPane();
root.setVgap(6);
root.setHgap(5);
```

```
root.setPrefWrapLength(250);
root.getChildren().add(new Button("Add"));
root.getChildren().add(new Button("Sub"));
root.getChildren().add(new Button("Mul"));
root.getChildren().add(new Button("Div"));
root.getChildren().add(new Button("Mod"));
```

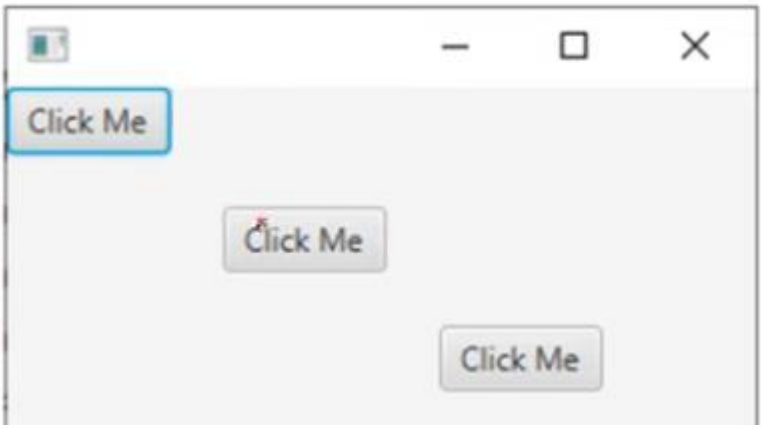
GridPane

GridPane Layout pane wllow us to add the multiple nodes in multiple rows and columns. It is seen as a flexible grid of rows and columns where nodes can be placed in any cell of the grid.



```
Label first_name=new Label("First Name");
Label last_name=new Label("Last Name");
TextField tf1=new TextField();
TextField tf2=new TextField();
Button Submit=new Button ("Submit");
GridPane root=new GridPane();
//root.setGridLinesVisible(true);
root.setVgap(10);
root.setHgap(10);
root.addRow(0, first_name,tf1);
root.addRow(1, last_name,tf2);
root.addRow(2, Submit);
```

hGap and VGap



BorderPane

Type	Property	Setter Methods	Description
Node	Bottom	setBottom()	Add the node to the bottom of the screen
Node	Centre	setCentre()	Add the node to the centre of the screen
Node	Left	setLeft()	Add the node to the left of the screen
Node	Right	setRight()	Add the node to the right of the screen
Node	Top	setTop()	Add the node to the top of the screen

```
BorderPane bPane = new BorderPane();

//Setting the top, bottom, center, right and left nodes to the pane
bPane.setTop(new TextField("Top"));
bPane.setBottom(new TextField("Bottom"));
bPane.setLeft(new TextField("Left"));
bPane.setRight(new TextField("Right"));
bPane.setCenter(new TextField("Center"));
```

JavaFX UI Controls

- Label
- Button
- RadioButton
- CheckBox
- ComboBox
- ListView
- TextField
- PasswordField
- Menu
- Table

Label

Label is a non-editable text control. A Label is useful for displaying text that is required to fit within a specific space.

Label Constructor

Label () Created an empty label

Label (String text) Create Label with supplied text

Label Method

```
Label lbl = new Label();
lbl.setFont(new Font("Times New Roman",32));
lbl.setText("This is simple text");
lbl.setTextAlignment(TextAlignment.CENTER);
//LEFT | CENTER | RIGHT |JUSTIFY
lbl.setTextFill(Color.RED);
lbl.setWrapText(true);
```

Setting an image for your Label

```
FileInputStream fis = new FileInputStream("pathofimage");
Image img = new Image(fis);
ImageView iv = new ImageView(img);
Label lbl = new Label("Enter Your Name here",iv);
```

Button

```
Button btn1= new Button("Click Me");
btn1.setText("This is text");
btn1.setWrapText(true);
btn1.setDisable(true);
btn1.setOnAction((event)->{
//code here
});
```

//Setting a button with image

```
FileInputStream fis = new FileInputStream("pathofimage");
Image img = new Image(fis);
ImageView iv = new ImageView(img);

Button btn1= new Button("Click Me",iv);
```

RadioButton

```
RadioButton rdo1= new RadioButton("English");
RadioButton rdo2= new RadioButton("Chemistry");
RadioButton rdo3= new RadioButton("Mathematics");
RadioButton rdo4= new RadioButton("Computer Science");
ToggleGroup group = new ToggleGroup();
rdo1.setToggleGroup(group);
rdo2.setToggleGroup(group);
rdo3.setToggleGroup(group);
rdo4.setToggleGroup(group);

//defining layouts
VBox h = new VBox();

//adding components to layout
```

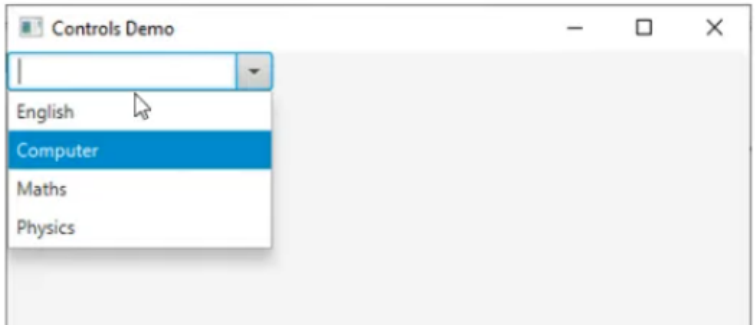
```
h.getChildren().addAll(rdo1,rdo2,rdo3,rdo4);
```

CheckBox

///coding of checkbox

```
CheckBox chk1= new CheckBox("English");
CheckBox chk2= new CheckBox("Chemistry");
CheckBox chk3= new CheckBox("Mathematics");
CheckBox chk4= new CheckBox("Computer Science");


//defining layouts
VBox h = new VBox();

//adding components to layout
h.getChildren().addAll(chk1,chk2,chk3,chk4);
```

ComboBox




```
ComboBox<String> cbo1= new ComboBox<>();
cbo1.getItems().add("English");
cbo1.getItems().add("Physics");
cbo1.getItems().add("Math");
cbo1.getItems().add("Java");
cbo1.setEditable(true);
```

ListView

```
ListView<String> cbo1= new ListView<>();
cbo1.getItems().add("English");
cbo1.getItems().add("Physics");
cbo1.getItems().add("Chemistry");
cbo1.getItems().add("History");
cbo1.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
```

TextField

```
TextField txt= new TextField();
txt.setMaxWidth(20);
```

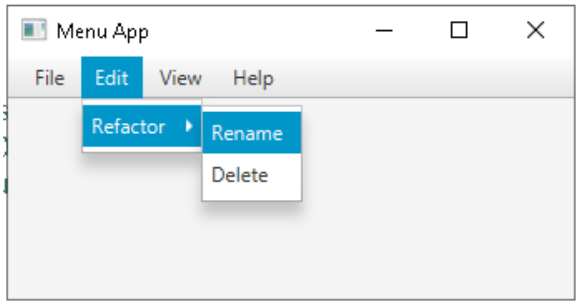
PasswordField

```
PasswordField txtPass= new PasswordField();
txtPass.setMaxWidth(20);
```

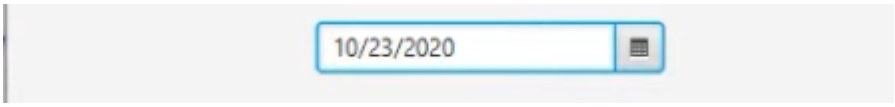
Menu

```
//menu bar
MenuBar menuBar = new MenuBar();

//menu
```

```
Menu file = new Menu("File");
Menu edit = new Menu("Edit");
Menu view = new Menu("View");
Menu help = new Menu("Help");

//binding menu to menuBar
menuBar.getMenus().add(file);
menuBar.getMenus().add(edit);
menuBar.getMenus().add(view);
menuBar.getMenus().add(help);

//menu item
file.getItems().add(new MenuItem("Open"));
file.getItems().add(new MenuItem("Save"));
file.getItems().add(new MenuItem("Close"));

//submenu
Menu refactor = new Menu("Refactor");
MenuItem rename = new MenuItem("Rename");
MenuItem delete = new MenuItem("Delete");
refactor.getItems().add(rename);
refactor.getItems().add(delete);
edit.getItems().add(refactor);

//add to layout
BorderPane root = new BorderPane();
root.setTop(menuBar);
```



DatePicker




```
DatePicker picker = new DatePicker();
```

Hyperlink

```
Hyperlink youtube= new Hyperlink("https://www.youtube.com/");
```

JavaFX Event Handling

To handle JavaFX event you need to **import javafx.event package**

```
Button btn1= new Button("Click Me");
btn1.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        System.out.println("Clicked");
    }
});


// lambda expression
btn1.setOnAction((event) -> {
      btn1.setText("You Clicked!");
});
```

For details:

https://docs.oracle.com/javafx/2/ui_controls/jfxpub-ui_controls.htm